💃 Robotic Arm and Chassis Coordinated Control (A1Pro Dancing)

1. Course Overview

In complex mobile manipulator applications, chassis movement and robotic arm joint motion often need close coordination. This course achieves a "dancing" effect of coordinated counter-movement between chassis and robotic arm by combining **robotic arm inverse kinematics (IK) real-time interpolation** with **chassis odometry closed-loop drive control**:

- **Forward Coordination Phase**: Chassis moves forward at constant speed for 1 meter, while the robotic arm smoothly extends backward through inverse solving, achieving dynamic spatial pose compensation for the end-effector;
- **Backward Coordination Phase**: Chassis reverses 1 meter, robotic arm synchronously pitches forward to reset.

🎯 Course Objectives

1. **Master multi-node coordinated startup process**: Understand the startup and coordination of low-level driver, inverse kinematics solver service, and coordinated control nodes.
2. **Master odometry closed-loop and travel projection algorithm**: Understand how to precisely map odometry integration to vehicle forward/reverse displacement using yaw angle projection.
3. **Understand IK-based spatial trajectory planning**: Master the pose definition of the robotic arm end-effector in Cartesian space and the conversion to joint angles.

---

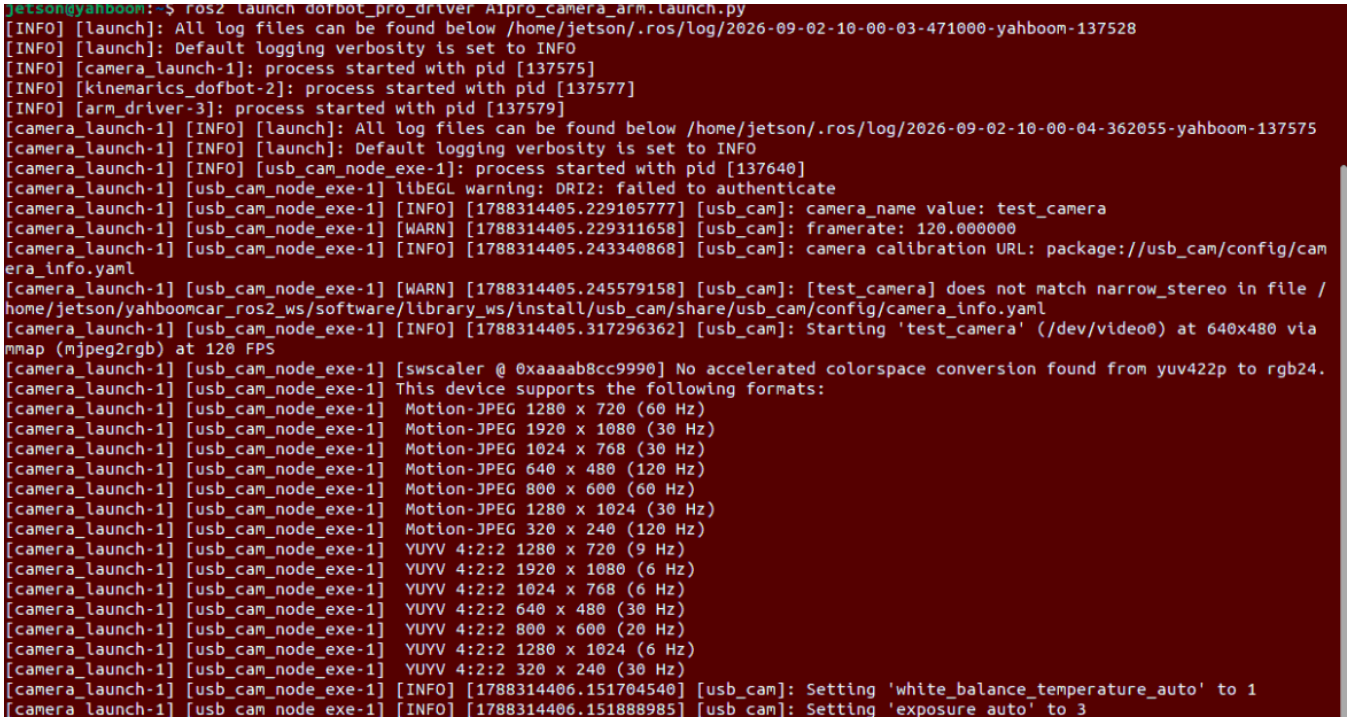
2. Preparation and Running Steps

Running this case requires starting the driver solver system and coordinated control nodes sequentially in terminals:

2.1 Start Inverse Solver and Low-Level Driver

Run in **Terminal 1** the launch file to start camera driver, KDL inverse solver node, and chassis driver with one command:

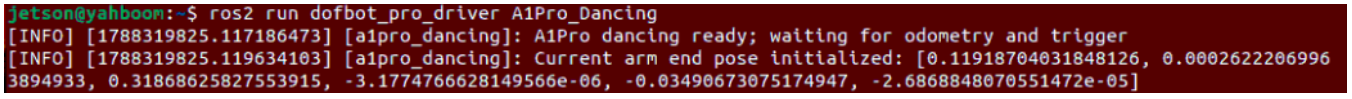
```
ros2 launch dofbot_pro_driver A1pro_camera_arm.launch.py
```



2.2 Start Coordinated Control Node

Start the coordinated control main program in **Terminal 2**:

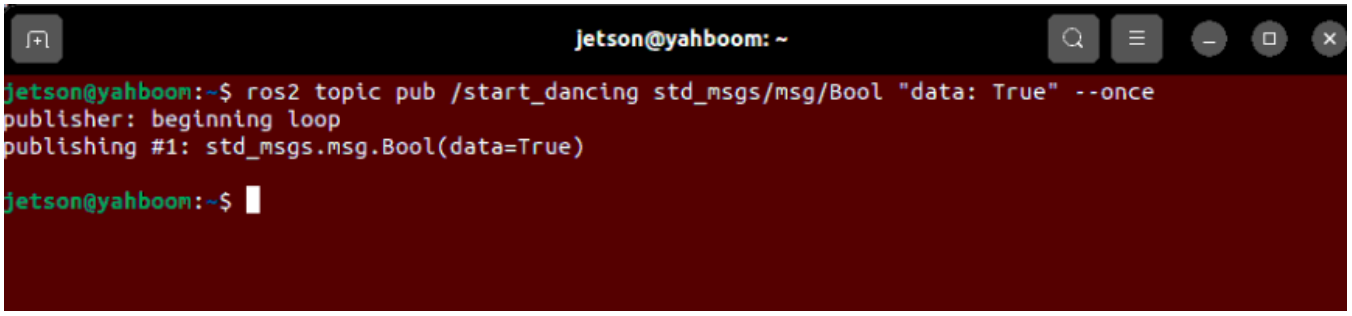
```
ros2 run dofbot_pro_driver A1Pro_Dancing
```



2.3 Trigger Coordinated Movement

Publish the startup topic in **Terminal 3** to trigger a forward motion:

```
ros2 topic pub /start_dancing std_msgs/msg/Bool "data: True" --once
```



Publish the trigger again for backward motion:

```
ros2 topic pub /start_dancing std_msgs/msg/Bool "data: True" --once
```

```
jetson@yahboom:~$ ros2 topic pub /start_dancing std_msgs/msg/Bool "data: True" --once
publisher: beginning loop
publishing #1: std_msgs.msg.Bool(data=True)

jetson@yahboom:~$ ros2 topic pub /start_dancing std_msgs/msg/Bool "data: True" --once
publisher: beginning loop
publishing #1: std_msgs.msg.Bool(data=True)

jetson@yahboom:~$
```

3. Coordinated Movement and Kinematics Effect Details

3.1 Motion Phase Description

1. **Phase 1 (Forward Coordination)**:
    - First publish `/start_dancing` trigger command;
    - Chassis moves forward at constant linear velocity of `0.1 m/s` for `1.0 meter`;
    - Robotic arm calls inverse solver, extending smoothly backward along the vehicle X axis by `10 cm`, forming a "vehicle forward, arm backward" dynamic spatial pose compensation effect;
    - After traveling 1.0 meter, chassis brakes smoothly, robotic arm precisely locks at the target extended pose.
2. **Phase 2 (Backward Coordination)**:
    - Publish `/start_dancing` command again;
    - Chassis reverses at constant linear velocity of `0.1 m/s` for `1.0 meter`;
    - Robotic arm moves smoothly forward, pitching reset by `10 cm`, returning to initial ready pose;
    - Cycles repeatedly, achieving neat and elegant coordinated dancing motion.

```
jetson@yahboom:~$ ros2 run dofbot_pro_driver A1Pro_Dancing
[INFO] [1788319825.117186473] [a1pro_dancing]: A1Pro dancing ready; waiting for odometry and trigger
[INFO] [1788319825.119634103] [a1pro_dancing]: Current arm end pose initialized: [0.11918704031848126, 0.0002622206996
3894933, 0.31868625827553915, -3.1774766628149566e-06, -0.03490673075174947, -2.6868848070551472e-05]
[INFO] [1788319917.907375026] [a1pro_dancing]: Starting forward stage: distance=1.00 m, speed=0.10 m/s
[INFO] [1788319929.006285832] [a1pro_dancing]: forward stage complete; next trigger runs reverse
[INFO] [1788319940.714524921] [a1pro_dancing]: Starting reverse stage: distance=1.00 m, speed=0.10 m/s
[INFO] [1788319951.801207409] [a1pro_dancing]: reverse stage complete; next trigger runs forward
```

3.2 Motion Parameter Configuration

Parameter Name	Default Value	Type	Function Description
`target_distance`	`1.0`	`float`	Single-phase chassis movement target distance (unit: meters)
`linear_speed`	`0.1`	`float`	Chassis movement linear velocity (unit: m/s)
`arm_delta_x`	`0.1`	`float`	Robotic arm end-effector extension displacement along X direction (unit: meters)
`control_rate`	`20.0`	`float`	Control loop publishing frequency (unit: Hz)
`motion_timeout`	`20.0`	`float`	Single-phase maximum timeout protection time (unit: seconds)

Parameter Name	Default Value	Type	Function Description
`odom_timeout`	`1.0`	`float`	Odometry data freshness timeout threshold (unit: seconds)

4. Core Source Code Deep Analysis

Source code path:

```
dofbot_pro_ws/src/dofbot_pro_driver/dofbot_pro_driver/A1Pro_Dancing.py
```

4.1 Node Initialization and FK to Get Initial End-Effector Pose

The node declares parameters, subscribes to `/odom_raw` odometry, creates `/get_kinemarics` client, places the robotic arm in initial pose, and computes the initial Cartesian spatial pose through forward kinematics (FK):

```
self.current_joints = [90.0, 150.0, 12.0, 20.0, 90.0, 0.0]
# Publish initial joint angles, smooth running time 2000ms
self.publish_arm(self.current_joints, 2000)
# Request FK forward kinematics to get initial end-effector Cartesian coordinates (x, y, z,
roll, pitch, yaw)
self.request_current_end_pose(self.current_joints)
```

4.2 IK Target Planning and Safety Limit Validation

In the method, requests are sent to the KDL inverse solver service based on the planned end-effector target coordinates, with coordinate frame mapping and software limit checks for each joint:

```
def solve_ik(self, target_pose, seed_joints):
    request = ArmKinemarics.Request()
    request.tar_x = float(target_pose[0])
    request.tar_y = float(target_pose[1])
    request.tar_z = float(target_pose[2])
    request.roll = float(target_pose[3])
    request.pitch = float(target_pose[4])
    request.yaw = float(target_pose[5])
    request.cur_joint1 = float(seed_joints[0])
    # ... set seed angles and call ik service ...
    request.kin_name = 'ik'

    # Convert joint 1 servo coordinate frame sign (180° - joint1)
    target_joints = [
        180.0 - response.joint1,
        response.joint2,
        response.joint3,
        response.joint4,
        response.joint5,
        30.0, # Gripper stays slightly open at 30°
```

```
    ]
    # Strictly validate each joint angle is within A1 Pro allowed physical safety range
    return target_joints
```

4.3 Odometry Yaw Projected Distance Calculation

To prevent minor angle yaw accumulation error from accumulating during vehicle travel, the node strictly projects displacement in the world frame onto the initial vehicle heading axis:

```
@staticmethod
def projected_distance(start_odom, current_odom, direction):
    dx = current_odom[0] - start_odom[0]
    dy = current_odom[1] - start_odom[1]
    heading = start_odom[2] # Initial heading angle Yaw
    return direction * (
        dx * math.cos(heading) + dy * math.sin(heading)
    )
```